ROS 2 for Robot Learning: A Complete Practical Tutorial

Technical Tutorial

2026-10-18

Contents

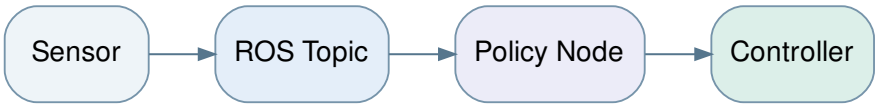
1. What Is ROS, and Why Use It in Robot Learning? . . . 2
2. ROS 1 vs. ROS 2 and Choosing a Distribution . . . 4
3. Core ROS 2 Concepts . . . 5
4. Installing ROS 2 Jazzy on Ubuntu 24.04 . . . 7
5. Hands-on: Writing a Robot Learning Policy Node in Python . . . 9
6. Recording Robot Learning Data with ROS 2 . . . 12
7. URDF, TF2, RViz, and ros2_control . . . 15
8. Connecting ROS to Isaac Lab / MuJoCo RL . . . 17
9. A Tron2 Architecture . . . 19
10. Common ROS Issues in Robot Learning . . . 21
11. A Recommended Learning Path . . . 22
12. Summary and Practical Recommendations . . . 23
13. Official Documentation and References . . . 23

Course home ·简体中文 ·PDF

Item	Details
Applicability	Ubuntu 24.04; ROS 2 Jazzy; Python examples
Documentation review date	2026-10-18
License	CC BY 4.0 ·Xiong Zeng and contributors

1. What Is ROS, and Why Use It in Robot Learning?

Chapter concept map



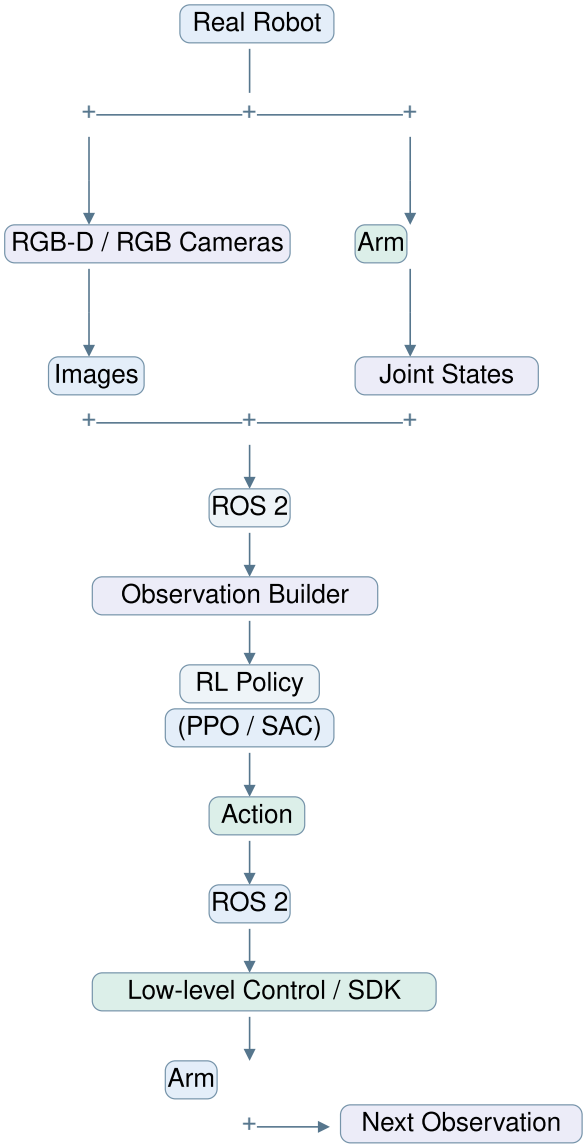
1.1 Definition of ROS

Despite its name, the Robot Operating System (ROS) is not an operating system in the sense of Ubuntu or Windows. It is a robotics software framework and ecosystem. Its central role is to allow separate robot software components to communicate, share information, and work together.

A robot may have cameras, a manipulator, an object pose estimator, a neural policy, and low-level controllers. The components can be written in different languages, run in different processes, or even live on different computers. ROS supplies a common messaging, communication, and tooling layer.

1.2 A typical robot learning system

Consider a Tron2 arm performing a screwdriver pick-up and rotation task:



The surrounding software systems have different responsibilities:

Component	Primary role in robot learning
ROS 2	Inter-module communication, system integration, transforms, recording, debugging
PyTorch	Neural-network training, loading, and inference
PPO / SAC	Reinforcement learning algorithms
Isaac Lab / MuJoCo	Physics simulation and RL training environments
ros2_control	Hardware abstraction, controller management, and command interfaces
MoveIt 2	Motion planning, kinematics, and collision checking
RViz 2	Visualization of robot state, sensors, and transforms
rosbag2	Recording and replaying ROS messages

The essential distinction is that ROS handles integration and communication, learning frameworks handle training

and inference, and controllers execute the resulting commands. They complement rather than replace one another.

1.3 Is ROS always required for robot learning?

No. If a task only involves Python and a physics simulator, ROS may be unnecessary.

Scenario	ROS required?	Reason
Parallel PPO training in Isaac Lab	No	Direct simulator APIs reduce overhead
SAC training in MuJoCo	No	Python environment and algorithm can interact directly
Offline Diffusion Policy / VLA training	No	Dataset loader plus PyTorch is sufficient
Controlling a single arm through its SDK	Not necessarily	Direct vendor SDK calls may be sufficient
Multi-camera + arm integration	Recommended	Simplifies modularity, monitoring, and synchronization
Teleoperation data collection	Recommended	Simplifies multimodal recording and replay
Real-robot deployment of RL policies	Recommended, not mandatory	Standardizes observation/action and device boundaries
Distributed or multi-robot systems	Recommended	Facilitates inter-process and inter-machine communication

> **Practical rule**: Keep the high-throughput simulated training loop outside ROS when possible. ROS is particularly useful around real-robot sensing, state estimation, data collection, device control, and deployment.

2. ROS 1 vs. ROS 2 and Choosing a Distribution

2.1 Main architectural differences

ROS 1 is the earlier ecosystem. ROS 2 was redesigned for contemporary robotics and offers better foundations for distributed discovery, QoS, lifecycle management, cross-platform support, and integration with real-time control systems.

Feature	ROS 1	ROS 2
Discovery	Typically depends on a ROS Master	Common DDS implementations support distributed discovery
Python API	`rospy`	`rclpy`
C++ API	`roscpp`	`rclcpp`
Bag recording	`rosbag`	`rosbag2`
Parameters	ROS Parameter Server	Parameters managed by individual nodes
QoS	More limited	Reliability, history, depth, deadlines, and more
Control framework	`ros_control`	`ros2_control`
Recommended for new projects	Generally no	Yes

Importantly, ROS 2 does **not** automatically make an application hard real-time. Determinism still depends on the OS, scheduling, hardware drivers, and system design.

2.2 Which ROS distribution should you use in 2026?

As of **October 2026**, ROS 2 **Lyrical Luth** has been released (May 22, 2026). The correct distribution nevertheless depends on the installed Ubuntu release and vendor-driver compatibility.

Ubuntu version	Common matching ROS 2 release	Support note
Ubuntu 22.04	Humble Hawksbill	LTS through May 2027
Ubuntu 24.04	**Jazzy Jalisco**	LTS through May 2029
Ubuntu 26.04	Lyrical Luth	LTS through May 2031

This tutorial targets Ubuntu 24.04 + ROS 2 Jazzy. If a hardware vendor only provides drivers for a specific ROS release, confirm driver compatibility before choosing a distribution.

3. Core ROS 2 Concepts

3.1 Node

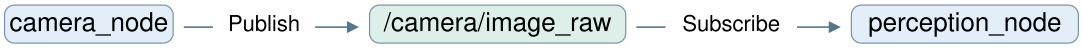
A Node is a basic ROS computational unit: a software component with a clear responsibility. A node is not necessarily a separate OS process; multiple nodes can be composed into one process.

```
ROS 2 System
|-- camera_node          # Acquire images
|-- robot_state_node     # Read joint states
|-- object_pose_node     # Estimate object 6D pose
|-- rl_policy_node       # Run policy inference
`-- controller_node      # Execute commands
```

The separation allows a perception component such as FoundationPose to be updated without rewriting the arm driver.

3.2 Topic: publish / subscribe

Topics are used for continuous, asynchronous data streams:



The publisher does not need to know its subscribers, and multiple subscribers may receive the same stream. Common robot learning topics include:

Example topic	Data
`/camera/color/image_raw`	RGB image
`/camera/depth/image_rect_raw`	Depth image
`/joint_states`	Robot joint state
`/object_pose`	Object pose
`/policy/action`	Action generated by the policy
`/tf`, `/tf_static`	Dynamic and static transforms

The main role of topics in robot learning is to carry data among perception, observation construction, the policy, and the controller. Actual topic names depend on the driver and namespace.

3.3 Message: data schema

A Message specifies the fields and types transmitted by a topic. Common messages include:

Message type	Purpose
`sensor_msgs/msg/Image`	Image
`sensor_msgs/msg/CameraInfo`	Camera calibration information and intrinsics
`sensor_msgs/msg/JointState`	Joint names, positions, velocities, and effort
`sensor_msgs/msg/Imu`	IMU measurements
`geometry_msgs/msg/Pose`	Position and orientation
`geometry_msgs/msg/PoseStamped`	Timestamped pose in a named frame
`geometry_msgs/msg/Twist`	Linear and angular velocity
`trajectory_msgs/msg/JointTrajectory`	Joint trajectory
`std_msgs/msg/Float64MultiArray`	A basic float array

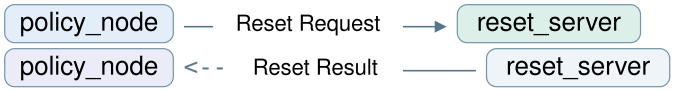
For example, `sensor_msgs/msg/JointState` includes:

```
header       # Timestamp and metadata
name         # Joint names, e.g. ['joint_1', 'joint_2']
position     # Joint positions
velocity     # Joint velocities
effort       # Joint torques or forces
```

Never assume that array indices have the same joint meaning across devices.

3.4 Service: request / response

A Service is useful for one-off operations that should return a response:



Typical uses include resetting a robot, querying device information, switching modes, or starting calibration. A service resembles a remote function call, but can still fail or time out.

3.5 Action: longer operations with progress feedback

An Action supports the workflow "submit goal -> receive progress -> get result". Examples include executing a manipulator trajectory, navigating to a goal, and running a multi-second grasp skill. Goals can be canceled.

Communication method	Best fit	Robot learning example
Topic	Continuous, asynchronous streaming	Images, joint state, policy actions
Service	One-off request and reply	Reset, query, mode switch
Action	Long operation with feedback and result	Execute a grasp or trajectory

For periodic closed-loop commands at, say, 50 Hz, one normally uses a topic or a direct in-process interface rather than creating a new Action every control tick.

3.6 Parameter: node configuration

Parameters configure nodes without changing their main code:

```
policy_node:
  ros__parameters:
    control_frequency: 50.0
    use_camera: true
    observation_dim: 32
    policy_path: "/home/robot/policy.pt"
```

Common settings include control frequency, camera names, model paths, reference frames, action scaling, and joint limits.

3.7 TF2: coordinate-frame management

A typical robot has many frames:

```
world
 `-- base_link
      |-- arm_base
      |    `-- ee_link
      |         `-- hand_base
      `-- head_link
           `-- camera_link
                `-- camera_optical_frame
```

Suppose FoundationPose estimates the pose of an object in the camera frame, `T(C,O)`. The manipulation policy needs the pose in the robot base frame:

Transform composition: `T(B,O) = T(B,C) x T(C,O)`

Here `B` means robot base, `C` means camera, and `O` means object. TF2 stores and queries transforms between frames, including time-dependent transforms.

TF2 does not calibrate a camera for you. The extrinsic transform `T(B,C)` must first come from URDF geometry, hand-eye calibration, or another valid calibration procedure. This is especially relevant when working with Tron2 head/wrist cameras and the offsets among yaw/pitch joints.

3.8 QoS: Quality of Service

QoS controls message delivery behavior:

Policy	Meaning
Reliability	`Reliable` vs. `Best Effort`
History / Depth	How messages are buffered
Durability	Whether retained data can be delivered to new subscribers
Deadline	Expected update interval and deadline monitoring

Camera streams often use `Best Effort` to avoid retransmitting stale frames. Some important state or command streams may use `Reliable`, but queues of outdated commands must still be prevented. Incompatible publisher/subscriber QoS can make a topic visible while delivering no data.

4. Installing ROS 2 Jazzy on Ubuntu 24.04

The commands below follow the official **Jazzy Ubuntu Debian package instructions**. Installation requires network access and administrator privileges. First check that another ROS installation will not conflict.

4.1 Check your system and configure repositories

```
lsb_release -a                 # Verify Ubuntu 24.04
locale                         # Verify a UTF-8 locale

sudo apt update
sudo apt install software-properties-common curl -y
sudo add-apt-repository universe

export ROS_APT_SOURCE_VERSION=$(curl -s \
  https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest \
  | grep -F 'tag_name' | awk -F\" '{print $4}')

curl -L -o /tmp/ros2-apt-source.deb \
  "https://github.com/ros-infrastructure/ros-apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-a
  ↪ pt-source_${ROS_APT_SOURCE_VERSION}.$(. /etc/os-release && echo
  ↪ ${UBUNTU_CODENAME:-${VERSION_CODENAME}})_all.deb"

sudo dpkg -i /tmp/ros2-apt-source.deb
```

If the system does not have a UTF-8 locale, see the official locale setup procedure, for example:

```
sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
```

4.2 Install ROS 2 and developer tools

```
sudo apt update
sudo apt install ros-jazzy-desktop ros-dev-tools
```

`ros-jazzy-desktop` includes RViz, demos, and common tools. For a non-GUI installation, `ros-jazzy-ros-base` is a smaller alternative.

4.3 Source the environment and verify

```
source /opt/ros/jazzy/setup.bash
echo $ROS_DISTRO               # Expected: jazzy
```

To source the environment automatically in new Bash sessions:

```
echo 'source /opt/ros/jazzy/setup.bash' >> ~/.bashrc
```

Avoid unintentionally sourcing multiple ROS distributions in the same shell.

4.4 First communication test

Terminal A:

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_cpp talker
```

Terminal B:

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_py listener
```

Expected output includes messages similar to `Publishing: Hello World` and `I heard: Hello World`, verifying Python/C++ interoperability.

4.5 Essential command-line tools

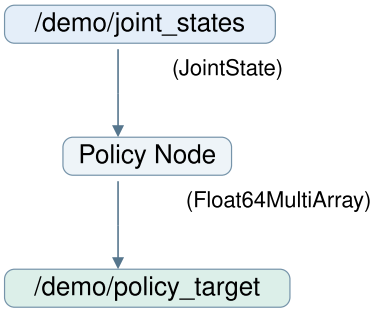
```
ros2 node list
ros2 node info /camera_node
ros2 topic list
ros2 topic echo /joint_states
ros2 topic type /joint_states
```

```
ros2 topic hz /joint_states
ros2 topic bw /camera/color/image_raw
ros2 topic info -v /joint_states
ros2 interface show sensor_msgs/msg/JointState
ros2 service list
ros2 action list
ros2 run tf2_ros tf2_echo base_link camera_link
```

The node/topic/frame names are examples. During real-robot debugging, `ros2 topic list`, `echo`, `hz`, and `info -v` are especially valuable for verifying message content, update rates, and QoS compatibility.

5. Hands-on: Writing a Robot Learning Policy Node in Python

We will build a minimal reproducible pipeline that **does not command any real motors**:



5.1 Create a workspace

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
```

Typical structure:

```
ros2_ws/
|-- src/      # Source code
|-- build/    # Build artifacts (generated)
|-- install/  # Installed packages (generated)
`-- log/      # Build logs (generated)
```

5.2 Create a Python package

```
cd ~/ros2_ws/src
ros2 pkg create \
    --build-type ament_python \
    --license Apache-2.0 \
    --node-name demo_policy \
    robot_learning_demo
```

The generated files resemble:

```
robot_learning_demo/
|-- package.xml
|-- setup.py
|-- setup.cfg
|-- resource/
`-- robot_learning_demo/
    |-- __init__.py
    `-- demo_policy.py
```

`package.xml` declares dependencies; `setup.py` defines Python installation and executable entry points; `demo_policy.py` contains the policy node.

5.3 Implement the policy node

Edit `~/ros2_ws/src/robot_learning_demo/robot_learning_demo/demo_policy.py`:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState
from std_msgs.msg import Float64MultiArray


class PolicyNode(Node):
    def __init__(self):
        super().__init__("policy_node")

        # Subscribe: virtual robot observation
        self.subscription = self.create_subscription(
            JointState,
            "/demo/joint_states",
            self.state_callback,
            10,
        )

        # Publish: demonstration action (not for real hardware)
        self.publisher = self.create_publisher(
            Float64MultiArray,
            "/demo/policy_target",
            10,
        )

    def state_callback(self, msg):
        q = list(msg.position)
        if not q:
            return

        # An illustrative mapping, not a trained RL policy
        action = [0.5 * x for x in q]

        command = Float64MultiArray()
        command.data = action
        self.publisher.publish(command)


def main(args=None):
    rclpy.init(args=args)
    node = PolicyNode()
    try:
        rclpy.spin(node)
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == "__main__":
    main()
```

The demonstration mapping is `a_t = 0.5 q_t`. It illustrates how observations become actions; it is **not** a trained or safe robot control policy. `Float64MultiArray` is chosen for simplicity; actual systems must define joint ordering, units, command semantics, limits, and timeout handling.

5.4 Declare dependencies and entry point

Add these dependencies to `package.xml` (do not duplicate existing entries):

```
<exec_depend>rclpy</exec_depend>
<exec_depend>sensor_msgs</exec_depend>
<exec_depend>std_msgs</exec_depend>
```

Check that `setup.py` includes the following `console_scripts` entry:

```
'demo_policy = robot_learning_demo.demo_policy:main'
```

5.5 Build and run

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
ros2 run robot_learning_demo demo_policy
```

The node now waits for state messages. Ensure Jazzy is sourced before building with `colcon`.

5.6 Publish simulated state and inspect output

In a new terminal, source ROS:

```
source /opt/ros/jazzy/setup.bash
```

Publish a virtual two-joint state at 5 Hz:

```
ros2 topic pub -r 5 \
  /demo/joint_states \
  sensor_msgs/msg/JointState \
  "{name: ['j1', 'j2'], position: [0.4, -0.2]}"
```

In another terminal:

```
source /opt/ros/jazzy/setup.bash
ros2 topic echo /demo/policy_target
```

Expected output:

```
data:
- 0.2
- -0.1
```

You have completed the minimal pipeline: publish state -> subscribe -> compute an action -> publish the action.

5.7 Replace the toy mapping with a PPO / SAC policy

Once a trained neural policy is available, replace:

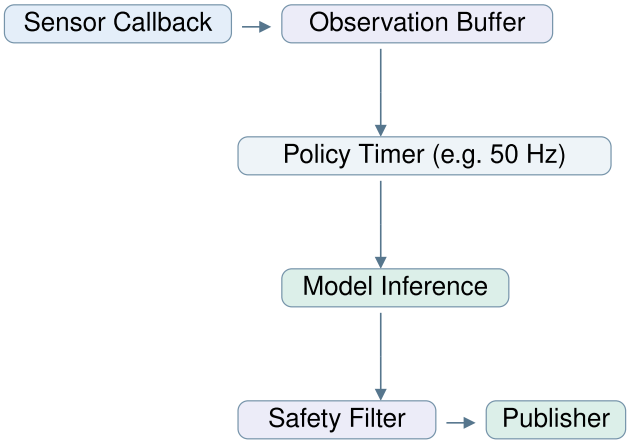
```
action = [0.5 * x for x in q]
```

with PyTorch inference (illustrative only):

```
with torch.inference_mode():
    action_tensor = policy(observation_tensor)
    action = action_tensor.cpu().numpy().tolist()
```

A real deployment must load model weights, construct the correct observation, normalize inputs, map joint order, scale actions, manage devices/dtypes, enforce limits, and handle failures.

It is often better to separate sensor callbacks from the fixed-frequency inference loop:

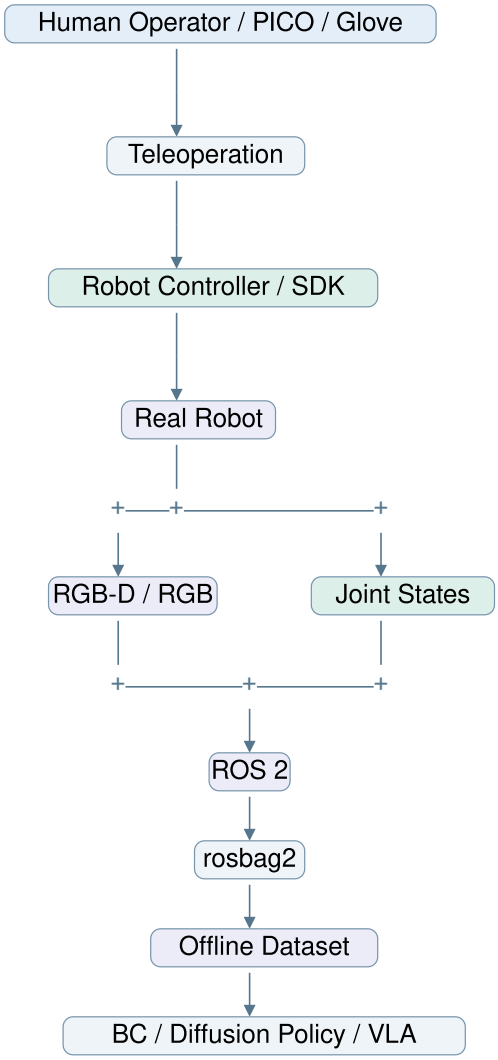


50 Hz is illustrative. The proper rate depends on inference cost, driver capabilities, and the task. Running inference unconditionally inside every subscription callback can lead to irregular execution, queue buildup, and latency.

6. Recording Robot Learning Data with ROS 2

6.1 Teleoperation data collection architecture

For example, collecting demonstrations by teleoperating Tron2 with PICO:



6.2 What is rosbag2?

rosbag2 is the ROS 2 tool for recording and replaying topic messages and related metadata. Common storage backends include SQLite3 and MCAP. It is useful for reproducing data collection sessions and diagnosing sensing or algorithm behavior.

6.3 Record and replay a session

Assume that these topics exist:

```
/joint_states
/camera/color/image_raw
/camera/depth/image_rect_raw
/tf
/tf_static
```

Record a selected set of topics:

```
ros2 bag record -o demo_001 \
  /joint_states \
  /camera/color/image_raw \
  /camera/depth/image_rect_raw \
  /tf \
  /tf_static
```

Stop with `Ctrl+C`, then inspect or replay:

```
ros2 bag info demo_001
ros2 bag play demo_001
```

Safety warning: Replaying a bag can republish control topics if those topics were recorded. Isolate the real hardware control interface or disconnect actuation before replaying such commands.

6.4 What should a robot learning dataset contain?

An imitation learning dataset can be written as:

`D = { (o_t, a_t) }_(t=0,...,T)`

Recommended fields include:

Category	Examples
Images	Head RGB, wrist RGB, head depth
Robot state	Arm/hand joint positions and velocities, end-effector pose
Action	Issued control commands and target joint positions/velocities
Time	Original timestamps from each data source
Calibration	Camera intrinsics/extrinsics, TF, URDF version
Episode metadata	Task, start/end time, success/failure
Optional	Tactile, torques, motor current, object pose

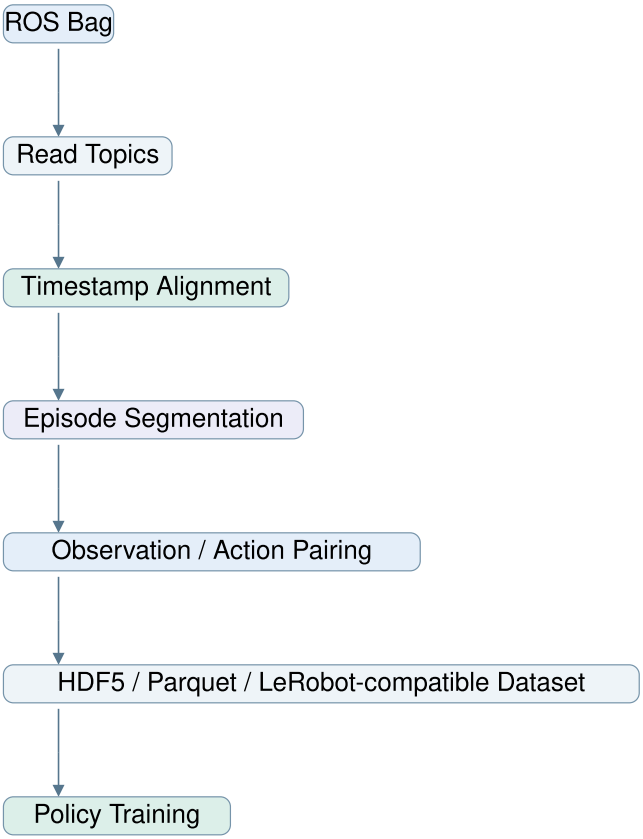
A crucial distinction is:

- **Commanded action:** A target generated by teleoperation or a policy.
- **Executed state:** What onboard sensors indicate the robot actually did.

They may differ due to delays, saturation, controller limits, or contact and must not be treated as interchangeable.

6.5 Time alignment and dataset conversion

A typical pipeline:



Example rates (not specifications for Tron2):

Modality	Illustrative rate
RGB / depth cameras	30 Hz
Robot joint state	100 Hz
Tactile sensing	100 Hz
Policy action	50 Hz

Choose a reference timeline, interpolate states, match image frames, or construct observation histories as appropriate. **Do not synchronize by the order in which ROS messages arrive; use meaningful timestamps and account for clock offsets.**

7. URDF, TF2, RViz, and ros2_control

7.1 URDF: robot model description

URDF (Unified Robot Description Format) describes links, joints, geometry, and inertial properties. For example:

```
Tron2
 |-- Shoulder Joint
 |-- Elbow Joint
 |-- Wrist Joint
```

Common XML elements include:

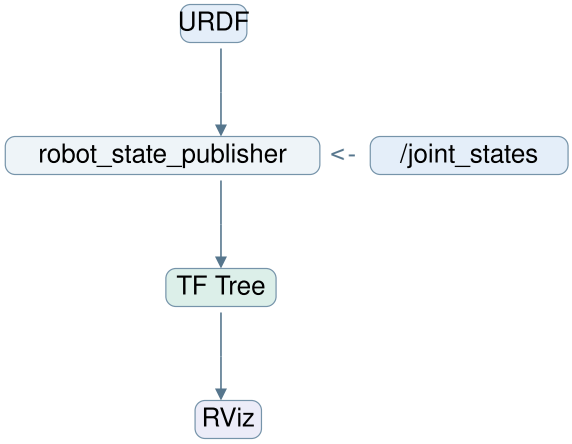
```
<link>       <!-- Rigid-body link -->
<joint>      <!-- Joint and connectivity -->
<visual>     <!-- Display geometry -->
```

```
<collision>  <!-- Collision geometry -->
<inertial>   <!-- Mass and inertia -->
```

URDF provides structure for visualization, kinematics, and conversion into simulation models, although simulation imports may need extra parameters and format conversion.

7.2 Relationship between URDF and TF2

URDF describes structure; TF2 manages coordinate transforms over time.



For Tron2, this helps inspect the end effector, head camera, and wrist camera. Incorrect URDF extrinsics will not be corrected by TF2 automatically.

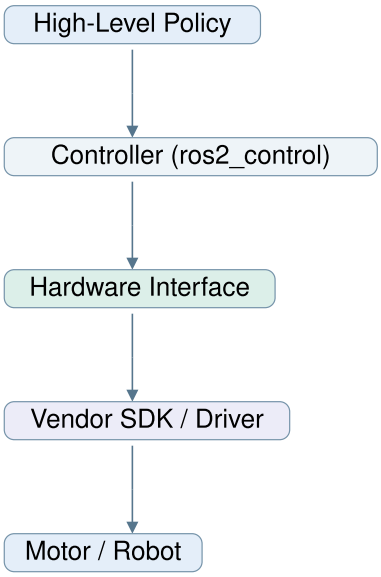
7.3 RViz 2: visualization

Start RViz:

```
rviz2
```

RViz can display the robot model, TF tree, camera images, point clouds, joint state, pose markers, and trajectories. With FoundationPose, it is useful for checking where the estimated object pose lies relative to the robot base.

7.4 ros2_control: connecting commands to hardware



A Hardware Interface maps low-level device read/write operations to the control framework. A Controller implements trajectory, position, velocity, or other command semantics.

Controller / concept	Purpose
Joint Trajectory Controller	Execute joint trajectories
Forward Command Controller	Forward command values to selected interfaces
Joint position/velocity command interfaces	Send commands as supported by configuration
Diff Drive Controller	Control a differential-drive mobile base

Install the packages:

```
sudo apt install \
  ros-jazzy-ros2-control \
  ros-jazzy-ros2-controllers
```

Once a compatible `controller_manager` is running:

```
ros2 control list_controllers
```

Installing ros2_control alone does not provide a Tron2 driver. You still need a compatible Hardware Interface or your own vendor-SDK adapter. A standalone ROS driver node without ros2_control is also possible.

8. Connecting ROS to Isaac Lab / MuJoCo RL

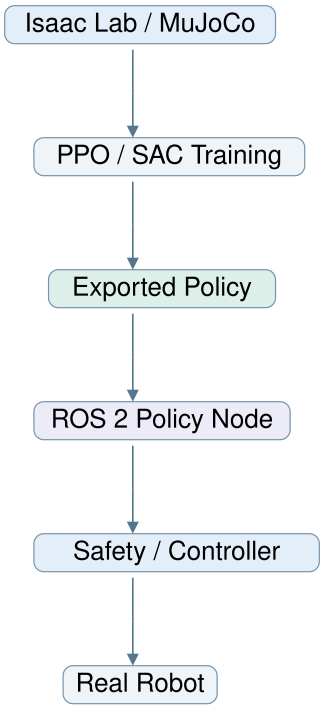
8.1 Simulated training usually needs no ROS

PPO/SAC training in Isaac Lab or MuJoCo often calls the environment API directly:

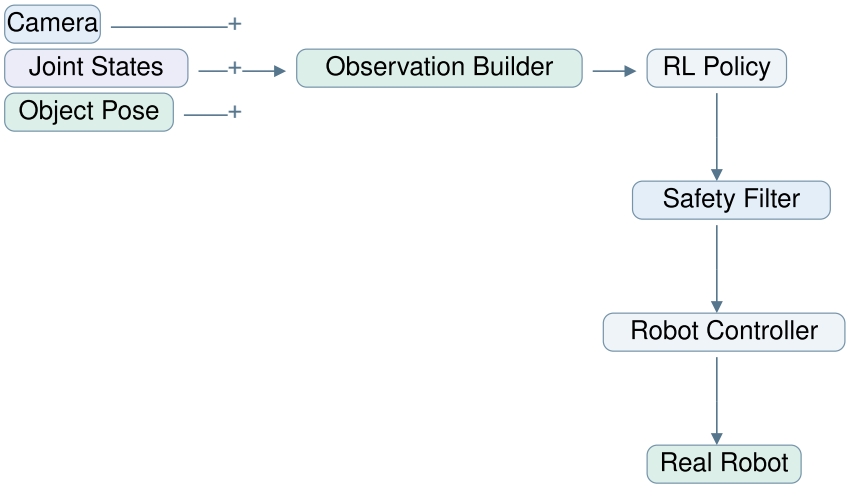
```
observations = env.reset()
for step in range(num_steps):
    actions = policy(observations)
    observations, rewards, dones, info = env.step(actions)
```

This is conceptual pseudocode, not an exact API signature for every Gymnasium or Isaac Lab version. GPU-parallel environments require high throughput; using a ROS topic at every simulation step usually adds unnecessary serialization and scheduling costs.

8.2 Real-world deployment needs integration



Typical execution architecture:



8.3 Consistent semantics matter more than transport

Item	Training	Real-robot requirement
Joint order	Simulator model order	Explicit mapping to driver order
Joint units	Often radians	Check radians, degrees, or vendor-specific units
Observation	Normalized state	Identical normalization and concatenation order
Action	Position, delta, velocity, etc.	Match the hardware control mode

Item	Training	Real-robot requirement
Frequency	Simulation timestep / decimation	Define policy and control periods
Frames	Simulation world/base	Define corresponding real-world frames

ROS standardizes communication but does not automatically fix joint ordering, camera calibration, normalization, dynamics mismatch, or latency.

8.4 Frequency and latency

Distinguish three clocks:

- `f_camera`: sensor acquisition rate.
- `f_policy`: neural-policy inference/update rate.
- `f_control`: low-level controller update rate.

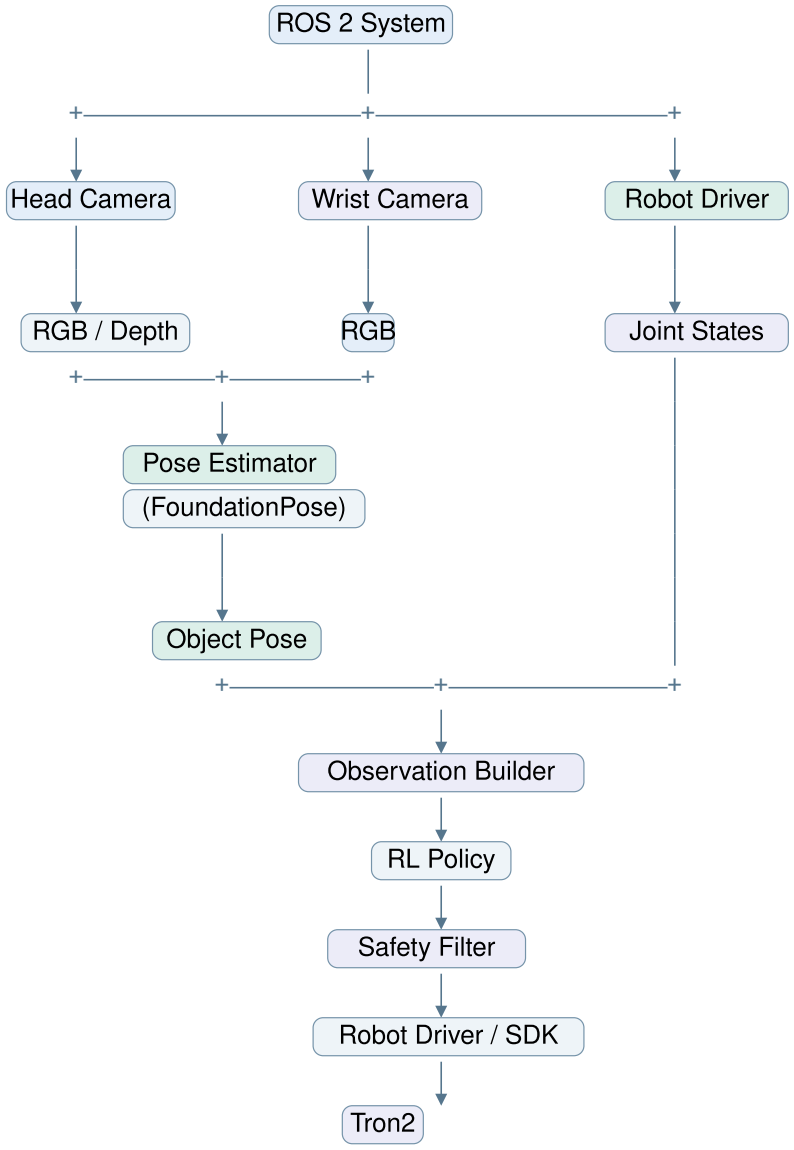
They need not be equal. For example, a 30 Hz camera, 50 Hz policy, and faster servo loop is a conceivable design, not a hardware requirement.

Python ROS 2 nodes do not automatically provide hard real-time execution. For robust control, separate GPU inference from servo control, place timing-critical tasks in appropriate C++/real-time threads, avoid blocking in the control path, and implement command timeouts, safety limits, and an emergency stop.

9. A Tron2 Architecture

Target task: **pick a screwdriver up from a table, carry it to a specified location, and rotate it**. The proposed system contains Tron2, a head RGB-D camera, a wrist RGB camera, FoundationPose, and a PPO/SAC policy.

9.1 Suggested ROS node architecture

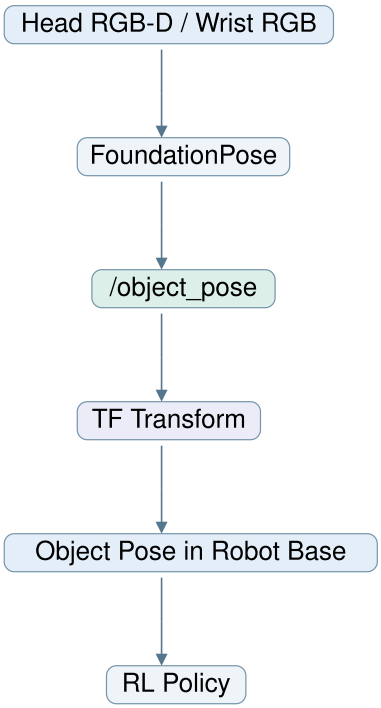


9.2 Suggested workspace layout

```
robot_learning_ws/
`-- src/
    |-- tron2_driver/
    |-- camera_driver/
    |-- robot_description/
    |-- pose_estimation/
    |-- observation_builder/
    |-- rl_policy/
    |-- safety_controller/
    |-- teleoperation/
    `-- data_recorder/
```

This is a proposed engineering layout, **not** a claim that the vendors distribute packages with these names. If a vendor only provides a Python/C++ SDK, wrap it in a ROS driver or adapter.

9.4 Integrating FoundationPose



The pose estimation node can be developed independently of the controller, provided its `PoseStamped` timestamps and `frame_id` are accurate. Real implementations should also handle occlusion, tracking loss/reinitialization, and temporal filtering.

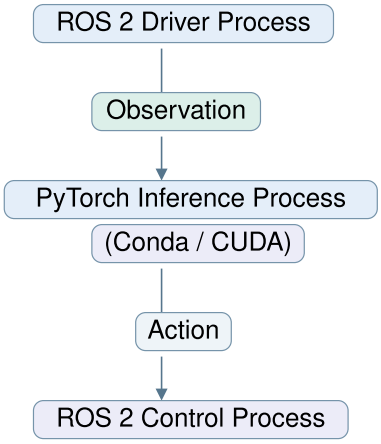
10. Common ROS Issues in Robot Learning

Symptom	Possible cause	What to check
`ros2 topic list` cannot see hardware	Driver not running, discovery or domain issue	Nodes, DDS, `ROS_DOMAIN_ID`
Topic exists but no messages arrive	Incompatible QoS or message types	`ros2 topic info -v`
Camera and joint states are misaligned	Different rates or clock domains	Timestamps, hardware sync, buffering
Object pose is biased	Calibration, TF chain, or frame mismatch	Extrinsics, optical frame convention
RL commands behave incorrectly	Joint order, normalization, or command mode mismatch	Compare training/deployment observation and action
End-to-end latency is high	Sensor, model, transport, SDK, controller delay	Measure each segment and queue occupancy
ROS conflicts with Conda	System Python vs. Conda ABI/dependency mismatch	Compatible environment or separate processes
Bag recordings are huge	Multiple high-bandwidth RGB-D streams	Compression, MCAP/storage settings, splitting
RViz robot model is wrong	URDF, JointState, or TF inconsistency	Inspect Robot Model and TF step by step

10.1 ROS 2 and Conda environments

Robot learning workflows commonly use Conda for PyTorch, CUDA, Isaac Lab, and RL libraries, while the apt-installed `rclpy` is tied to the system Python environment. Running `import rclpy` inside an arbitrary Conda environment may trigger ABI or shared-library problems.

A robust architecture is:



Processes can communicate through ROS topics, dedicated IPC, or shared memory, rather than requiring every dependency in the same Python environment.

10.2 Safety boundary for real hardware

A learned policy output is not a trusted low-level motor command. Real deployments need joint limits, velocity/acceleration bounds, command timeouts, stopping and emergency-stop mechanisms, disconnection fallback behavior, and initial testing at low speed without payload or contact. ROS is not a replacement for a hardware safety system.

11. A Recommended Learning Path

Stage	Topics	Demonstrable milestone
1	Nodes, topics, messages	Inspect and explain the robot's topics
2	`rclpy`, publishers, subscribers	Run a custom Python policy node
3	rosbag2	Record and replay a complete episode
4	URDF, TF2, RViz	Verify robot and camera coordinate frames
5	Robot SDK, ros2_control	Safely read state and send test targets
6	PyTorch + ROS	Deploy PPO/SAC inference on a robot
7	QoS, synchronization, latency	Quantify sensor-to-action end-to-end latency

For researchers already comfortable with Python, control, and robot learning, the most efficient path is to learn what is needed for **data collection -> policy deployment -> reproducible experiments**, rather than covering every ROS subsystem in isolation.

12. Summary and Practical Recommendations

The value of ROS 2 in robot learning can be grouped into three areas:

Area	ROS responsibilities	Typical application
Data collection	Gather and record camera, state, and teleoperation streams	BC, Diffusion Policy, VLA
Policy deployment	Connect observations, policy inference, and controllers	PPO, SAC, Sim2Real
System integration	Frame transforms, device communication, visualization, diagnostics	visual feedback, real-robot experiments

For a **Tron2** research platform, prioritize five skills:

1. **Topics + rclpy**: Connect state and camera observations to the RL policy.
2. **rosbag2**: Record teleoperation demonstrations for offline datasets.
3. **TF2 + URDF**: Maintain the head-camera, wrist-camera, end-effector, and object frames.
4. **Robot SDK / Hardware Interface**: Control Tron2 reliably.
5. **Sim2Real deployment**: Transfer policies trained in Isaac Lab / MuJoCo to a physical closed-loop system.

The goal is not to put more ROS into training. It is to build an observable, reproducible, maintainable sensor -> policy -> controller -> robot pipeline.

13. Official Documentation and References

1. ROS 2 Jazzy Tutorials
2. ROS 2 Jazzy Ubuntu Debian Package Installation
3. Python Publisher / Subscriber Tutorial
4. ROS 2 TF2 Tutorials
5. ros2_control Documentation
6. MoveIt 2 Documentation
7. rosbag2 GitHub Repository
8. ROS 2 Distribution Support Information
9. ROS 2 Lyrical Release Records

Document note: This tutorial is educational and intended for system design. The sample action topic is for simulated testing only. All real-robot integrations must be checked against vendor documentation and safety requirements.